

Workshop Manager for Garages

Job cards, technicians, parts stock, invoicing, service reminders and reports for an auto repair shop

IN ONE SENTENCE

A front-office system for a car repair workshop: job cards with labour and parts, automatic stock, sound invoices and service-due lists.

BUILT WITH

React 19, TypeScript, Vite, Recharts; production: Node 22, Fastify 5, PostgreSQL, Docker

What this guide covers

- 1 What the project is
- 2 The business problem it solves
- 3 The client's requirements
- 4 How the solution is designed
- 5 How to build it from scratch, step by step
- 6 Testing, deployment and known limits

Contents

1. What this project is	3
2. The business problem	4
3. Client requirements	5
4. How the solution is designed	7
5. Build it from scratch, step by step	9
6. Testing, deployment and known limits	13

1. What this project is

An independent garage runs on job cards: a car comes in, the service advisor writes up the work, technicians do it, parts come off the shelf, and the customer pays at collection. This project replaces the paper and spreadsheets with a **workshop board**, **job cards** with labour, parts and one-click service packages, **parts stock that tracks itself**, **invoices**, a **customer and vehicle history** with a service-due list, and **reports**.

The repository holds **two versions**: the root folder is a client-side demo, and [garage-workshop-manager-production-grade](#) runs on PostgreSQL with staff logins, legally sound invoices (frozen when issued, gapless numbers, void-and-reissue) and a stock ledger that explains every stock figure.

At a glance	
Type	Staff web application for a workshop
Users	Service advisor (front desk), technicians, owner/admin
Stack	React 19, TypeScript, Vite, Recharts; production: Node 22, Fastify 5, PostgreSQL, Docker
Demo data	6 months: ~160 customers, ~1,200 jobs, 3 technicians, 24 stock items

Who uses it

User	What they do with it
Service advisor	Books cars in, writes job cards, adds labour, parts and packages, completes jobs, takes payment, chases service-due customers.
Technician	Sees the board and job cards; moves jobs between In progress, Waiting parts and Ready. Cannot complete, cancel or price.
Owner / admin	Settings (workshop details, tax, labour rate, service interval), technicians, staff logins, reports, activity log.

Two versions in this repository

	Demo version (root folder)	Production version (...-production-grade folder)
Data	Browser localStorage	PostgreSQL shared by desk and workshop floor
Access	Anyone who opens the page	Logins: admin, service advisor, technician
Invoices	Recalculated from the job	Frozen when issued (totals, tax rate, customer, vehicle, workshop details); gapless numbers; void and reissue, never edit
Stock	A number jobs change	Every movement recorded (job, delivery, stocktake, opening balance)
Lists	All loaded in the browser	Server-side search and paging
Accountant	—	Invoice register CSV (issued, paid, voided) for any period

2. The business problem

Small workshops lose money in small ways: labour not written down, parts used but not billed, invoices edited after the fact, stock counts that never match the shelf, and regular customers who forget their next service. The owner has no view of technician utilisation or parts margin.

The client wants a **simple front-office system** that captures every job, keeps stock right automatically, produces invoices an accountant accepts, and brings customers back for servicing.

Pain point today	What it costs the business	How this project solves it
Paper job cards	Labour and parts not billed	Job cards with labour lines (hours x rate), parts lines and service packages; live totals
Stock counts wrong	Parts run out; theft unnoticed	Stock moves automatically with jobs; (production) every movement in a ledger
Invoices changed after issuing	Accounting and tax problems	(Production) invoices frozen when issued; corrections by void and reissue with gapless numbers
Customers forget services	Lost repeat work	Service-due list of cars not seen for N months and not already booked
No performance view	Owner guesses	Reports: revenue split, job value, gross profit, parts margin, turnaround, technician utilisation

Value to the business

- Every hour and every part billed.
- Clean books for the accountant.
- More repeat business from service reminders.

3. Client requirements

The brief below is written the way a client would hand it to a developer: what the system must do (functional requirements), how well it must do it (non-functional requirements), and what was explicitly left out. Every item below is implemented in the repository; section 6 explains how the project is tested.

Functional requirements

ID	Area	Requirement
FR-01	Workshop board	Today's cars by stage (Booked → In progress → Waiting parts → Ready) with one-click moves; overdue bookings; next 7 days; KPIs (cars in the shop, cash waiting at collection, unpaid invoices, parts to reorder).
FR-02	Job cards	Pick or add customer and vehicle on the spot; labour lines, part lines and one-click service packages (e.g. "Front brake pads" = labour + parts).
FR-03	Job cards	Live totals with discount and tax, gross profit, status history, technician assignment.
FR-04	Parts	Adding a part takes it off the shelf; editing quantity adjusts it; cancelling returns it; warnings when using more than in stock; receive stock; reorder levels; no duplicate SKUs; stock value at cost.
FR-05	Invoices	Issued when a job is completed; printable/PDF from the browser; mark as paid (cash, card, bank transfer); paid jobs locked. Production: frozen documents, gapless numbers, void and reissue, invoice register CSV.
FR-06	Customers	Search by name, phone or plate; vehicles and full service history; Service due list for reminder calls.
FR-07	Reports & settings	Revenue by week (labour vs parts), average job value, gross profit, parts margin, turnaround, unpaid invoices, technician utilisation, top services; every chart has a table view.
FR-08	Reports & settings	Workshop details, currency and locale, tax %, labour rate, service interval, technicians, backup/restore (demo).

Non-functional requirements

ID	Quality	Requirement
NFR-01	Money	Whole cents; tax rounded once on the net amount; discounts can't exceed the subtotal.
NFR-02	Stock correctness	Stock changes are the difference between a job's old and new parts, so saving twice never double-counts; cancelled jobs hold no stock; (production) stock equals the sum of its movements.
NFR-03	Security (production)	Passwords hashed with bcrypt; accounts lock after 5 failures; only hashes of session tokens are stored; sessions expire.
NFR-04	Security (production)	Every change needs a CSRF token and a same-site Origin; strict security headers (Content-Security-Policy, HSTS behind https).
NFR-05	Security (production)	Every input validated on the server (zod); all SQL parameterised; rate limits on the API and tighter ones on login.
NFR-06	Security (production)	An activity (audit) log of logins and every business action, viewable by admins.

ID	Quality	Requirement
NFR-07	Integrity (production)	Four jobs completed at once get four consecutive invoice numbers; paid invoices can't be voided; one live invoice per job; customer, vehicle and job created together or not at all; plates matched ignoring spaces and case; customers with history can't be deleted; conflicting edits refused.
NFR-08	Operations (production)	Runs as one Docker image next to PostgreSQL; health checks <code>/healthz</code> and <code>/readyz</code> ; database migrations run on start-up safely with several instances.
NFR-09	Operations (production)	Structured JSON logs with a request id (returned in the <code>x-request-id</code> header); cookies and tokens are redacted.

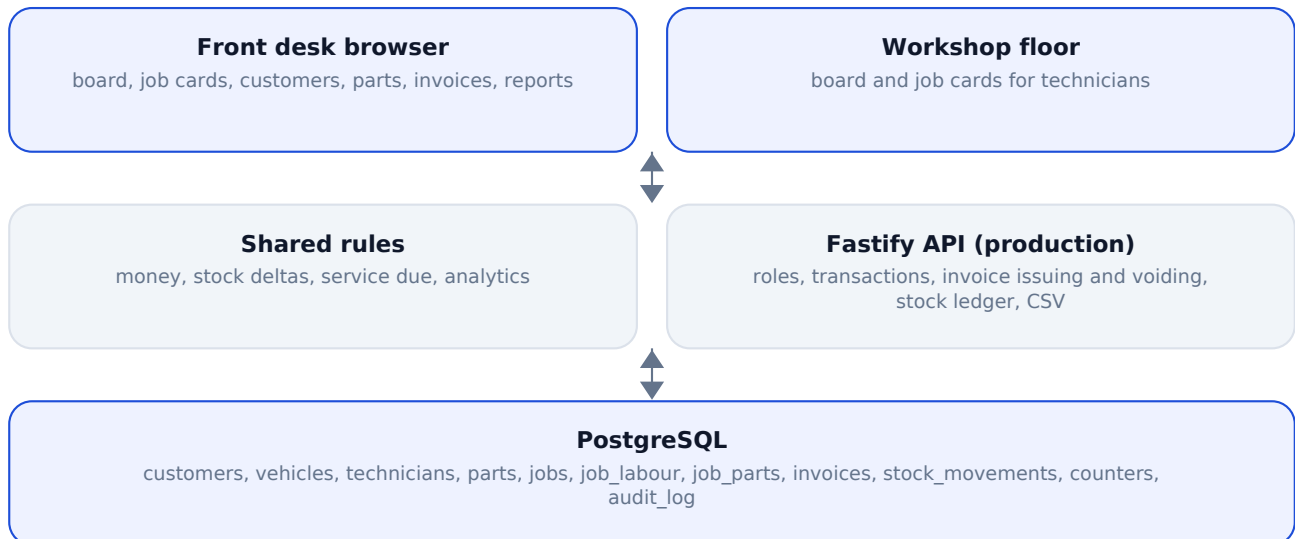
Out of scope for this release

- Part payments, deposits and card terminals.
- Refunds of paid invoices (done in the accounting system).
- SMS/email reminders (the Service due list shows whom to contact).

4. How the solution is designed

Business rules (money, stock differences, dates, analytics) are pure functions. The demo applies them to a localStorage store; the production server applies the same functions inside database transactions and stores issued invoices as frozen JSON documents.

Architecture



Technology choices

Part	Choice	Why
Web	React 19, TypeScript, Vite, Recharts	Fast desk UI; printable invoice page.
Rules	Pure TypeScript (<code>src/lib</code> , <code>shared/</code>)	Tested once, used by browser and server.
Server (prod)	Node 22 + Fastify 5 + PostgreSQL	Transactions for completing jobs and stock; constraints for invoices.
Tests	Vitest, Playwright	Money and stock maths; concurrency; browser workflows.

Data model

Table / file	What it holds
customers / vehicles	Customers and their cars (plate unique ignoring spaces/case), last service date.
technicians / parts	Technicians; parts with SKU, cost, price, stock and reorder level.
jobs / job_labour / job_parts / job_status_events	Job cards with status, technician, discount; labour and part lines; status history.
invoices	Number (gapless), frozen <code>document</code> JSON, issued/paid/voided times, void reason; one live invoice per job; paid and void can't both be set.
stock_movements	Every stock change with reason: opening, receive, adjust or job.
counters / settings / users / sessions / audit_log	Number sequences, workshop settings, staff and activity (production).

Key business rules

- **Totals:** labour (hours × rate) + parts, minus discount (never below zero), plus tax rounded once on the net.
- **Stock delta:** when a job is saved, stock changes by (new parts – old parts); cancelled jobs return everything.
- **Service due:** last service older than the interval (months) and no upcoming booking.
- **Invoice immutability (production):** once issued, work and prices can't change (notes and technician can); a correction voids the old invoice (kept) and issues a new number.

Folder layout

```
Root (demo):  src/lib (money, stock, dates, analytics) · store/ · pages/ · data/ (generator)
garage-workshop-manager-production-grade/:  shared/ · server/ (Fastify, migrations, tests) ·
web/ · docs/
```

5. Build it from scratch, step by step

Part A (steps 1–6) builds the demo that proves the workflow with the garage; Part B (steps 7–13) makes invoices and stock audit-proof for real use.

Before you start

- Node.js 22+; PostgreSQL and Docker for Part B.

Step 1. Part A · Scaffold and model

```
npm create vite@latest garage-workshop-manager -- --template react-ts
cd garage-workshop-manager && npm i react-router-dom recharts && npm i -D vitest playwright-core
```

1.1 Types for Customer, Vehicle, Technician, Part, Job (labour and part lines), Invoice, Settings; money in cents.

Done when: The app runs with routing.

Step 2. Part A · Money, stock and dates rules

2.1 `money.ts` (line totals, discount cap, tax once), `stock.ts` (delta between old and new job parts), `dates.ts`, `analytics.ts` (reports) with unit tests.

Done when: `npm test` passes.

Step 3. Part A · Demo data

3.1 Seeded 6-month generator: customers, vehicles, jobs through every stage, stock receipts, invoices; a test checks consistency (stock never negative, paid jobs invoiced).

Done when: Demo data test passes.

Step 4. Part A · Board and job cards

4.1 Workshop board with stage columns and one-click moves; job card editor with labour, parts, packages, live totals, gross profit, status history.

Done when: Adding a part to a job reduces stock; cancelling returns it.

Step 5. Part A · Invoices, customers, parts, reports

5.1 Invoice on completion with print view; mark paid; customer search and history; Service due list; parts receive/reorder; reports with table views; settings with backup/restore/reset.

Done when: A job goes from booking to paid invoice in the browser.

Step 6. Part A · Tests and publishing

6.1 Playwright smoke test of real workflows on desktop, phone and dark mode; static deploy workflow.

Done when: Tests pass.

Step 7. Part B · Set up the workspace

One repository with a server, a web app and shared code used by both.

- 7.1 Create an npm workspace with `server` and `web` packages and a `shared` folder that both import (types, zod schemas and business rules).

```
mkdir garage-workshop-manager-production-grade && cd garage-workshop-manager-production-grade
&& git init
npm init -y          # then add "workspaces": ["server", "web"] to package.json
npm create vite@latest web -- --template react-ts
mkdir -p server/src server/migrations shared
npm i -w server fastify @fastify/cookie @fastify/helmet @fastify/rate-limit @fastify/static pg
zod
npm i -D -w server typescript tsx vitest esbuild @types/node @types/pg
npm i -w web @tanstack/react-query react-router-dom
```

- 7.2 Turn on TypeScript `strict` everywhere. Add root scripts: `dev`, `build`, `typecheck`, `test`, `migrate`, `seed:demo`, `smoke`.

Done when: `npm run typecheck` passes on the empty project.

Step 8. Part B · Configuration, database connection and migrations

- 8.1 `server/src/config.ts`: read environment variables with a zod schema (`DATABASE_URL`, `PUBLIC_URL`, `COOKIE_SECURE`, `TRUST_PROXY`, `SESSION_TTL_HOURS`, `RATE_LIMIT_PER_MIN`, `TIMEZONE`) and refuse to start with a clear message when one is wrong.
- 8.2 `server/src/db.ts`: a `pg` connection pool with a statement timeout, and a `tx(fn)` helper that runs `fn` inside `BEGIN/COMMIT` and rolls back on any error.
- 8.3 `server/src/migrate.ts`: apply `migrations/*.sql` in name order, each in its own transaction, recorded in a `schema_migrations` table, under a PostgreSQL advisory lock so several app servers can start at the same time.

```
createdb app_dev
export DATABASE_URL=postgres://postgres:postgres@localhost:5432/app_dev
npm run migrate
```

Done when: Running `npm run migrate` twice applies the migrations once and then reports the database is up to date.

Step 9. Part B · Schema for sound invoices and stock

- 9.1 Tables as in the data model, with `counters` for gapless numbers and the invoice constraints:

```
number text NOT NULL UNIQUE, document jsonb NOT NULL, paid_at timestamptz, voided_at
timestamptz,
CONSTRAINT paid_not_void CHECK (NOT (paid_at IS NOT NULL AND voided_at IS NOT NULL))
CREATE UNIQUE INDEX invoices_one_live_per_job ON invoices (job_id) WHERE voided_at IS NULL;
reason text NOT NULL CHECK (reason IN ('opening', 'receive', 'adjust', 'job')) --
stock_movements
```

Done when: Migrations apply; constraints reject a paid-and-void invoice.

Step 10. Part B · Logins, roles and request security

The same proven foundation every production app here uses.

- 10.1 **Passwords:** hash with Node's built-in `scrypt` (memory-hard, no native add-on) and compare in constant time. For an unknown email, verify against a dummy hash so response time doesn't reveal which emails exist. Lock the account for 15 minutes after 5 wrong passwords.
- 10.2 **Sessions:** a random 32-byte token in an `httpOnly`, `SameSite=Lax` cookie (`Secure` in production). Store only the token's SHA-256 with a CSRF token, expiry and last-seen time; expire idle sessions.
- 10.3 **Every request:** a Fastify `onRequest` hook loads the session, refuses writes whose `Origin` is another site, and requires the `x-csrf-token` header on every POST/PUT/DELETE.
- 10.4 **Headers and limits:** Helmet with a strict Content-Security-Policy; `@fastify/rate-limit` keyed by the signed-in session (or IP for visitors), with tighter limits on login.
- 10.5 **Roles:** service advisor (front desk: jobs, customers, parts, prices, invoices, payments, reports), technician (board and job cards; can't complete, cancel or price) and admin (also settings, technicians, logins, activity log). A `requireUser(...roles)` pre-handler protects each route; the web app hides what a role can't use, but the server is what enforces it.
- 10.6 **Audit log:** an `audit(...)` helper writes who did what, to which record, from which IP, for every business action and every login.
- 10.7 **CLI:** `create-admin` to make the first admin (password from an environment variable or a hidden prompt).

Done when: API tests prove: wrong passwords lock the account, a write without the CSRF token is refused, each role gets 403 on routes it may not use.

Step 11. Part B · Jobs, invoices and stock API

- 11.1 Create customer + vehicle + job in one transaction (duplicate plate → nothing saved).
- 11.2 Save job lines with optimistic locking; apply stock deltas as `stock_movements` in the same transaction; part cost comes from the parts list, not the browser.
- 11.3 Complete job: next invoice number from `counters`, freeze the document (totals, tax rate, customer, vehicle, workshop details), lock prices. Void: keep the old invoice with a reason and issue a new number. Paid invoices can't be voided.
- 11.4 Invoice register CSV for a period; server-side search and paging; reports from the shared analytics.

Done when: Tests: four simultaneous completions → four consecutive numbers; later tax changes leave issued invoices unchanged; stock equals the sum of its movements.

Step 12. Part B · Test it end to end

- 12.1 **Unit tests** (Vitest) for every pure business rule in `shared/`.
- 12.2 **API tests** against a real PostgreSQL database: a global set-up drops and recreates the test database (only if its name contains `test`), migrates it and loads the demo data. Tests sign in as each role and call the real HTTP API.
- 12.3 **Browser smoke test** (`web/scripts/smoke.mjs` with Playwright): build, start the real server on a fresh database, then run a job from booking to paid invoice, a void and reissue, a stock delivery, and each role's permissions, Fail on any script error, and save screenshots.

```
npm run typecheck && npm test  
npm run build && npm run smoke
```

Done when: All three suites pass locally.

Step 13. Part B · Package, automate and document

- 13.1 A multi-stage `Dockerfile`: build everything, then a slim runtime image with production dependencies only, running as a non-root user, with a `HEALTHCHECK` on `/readyz`.
- 13.2 `docker-compose.yml` with PostgreSQL and the app; `.env.example` explaining every setting.
- 13.3 GitHub Actions: typecheck, all tests with a PostgreSQL service container, build, the browser smoke test, a dependency audit, and a job that builds the Docker image and checks it starts and answers.
- 13.4 `README.md` for users and `docs/OPERATIONS.md` for whoever runs it (https, backups, monitoring, upgrades).

Done when: CI is green on GitHub and `docker compose up -d --build` gives a working app.

6. Testing, deployment and known limits

How it is tested

- **Demo:** unit tests for money, stock, dates, analytics and demo-data consistency; browser test of real workflows on desktop, phone and dark mode.
- **Production:** invoices match job totals, are frozen and gapless, void-and-reissue works, paid can't be voided; stock ledger and returns; technicians can't complete or price; CSRF; conflicts; all-or-nothing creation; plate matching; reports equal shared statistics; full-stack browser test.

```
# demo
npm test && npm run build && npm run smoke
# production
npm run typecheck && npm test && npm run build && npm run smoke
```

Deploying and running it

- **Demo:** static hosting (Pages workflow started by hand, or Netlify/Vercel/Cloudflare Pages).

```
cd garage-workshop-manager-production-grade
cp .env.example .env          # POSTGRES_PASSWORD, PUBLIC_URL, TIMEZONE
docker compose up -d --build
docker compose exec app node dist/cli/create-admin.js --email owner@yourgarage.com --name
"Owner Name"
# demo: docker compose exec -e ALLOW_DEMO_SEED=1 app node dist/cli/seed-demo.js (admin@,
advisor@, tech@demo.local)
```

Known limits

- One payment per job (no deposits or part payments); no card terminal.
- Paid invoices can't be voided in the app (refunds in accounting).
- No SMS/email reminders.
- Rate limits per server instance.

Sensible next steps

- SMS/WhatsApp "your car is ready" and service reminders.
- Supplier parts catalogues and ordering.
- Accounting integration (QuickBooks/Xero); online booking from the garage website.